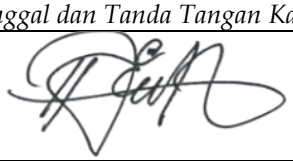
SOAL UJIAN TENGAH SEMESTER GANJIL TA.2026/2027
PELAKSANA AKADEMIK MATA KULIAH (PAMK)
JAKARTA GLOBAL UNIVERSITY (JGU)

Mata Kuliah	Framework Based Programming	Waktu	5 Days
Dosen	Halimatuz Zuhriyah, S.Kom., MIT	SifatUjian	Project
Jurusan	Teknik Informatika	Waktu Kuliah	Pagi | Karyawan
Hari/Tgl.	Senin, 2 Nov 2026	Jam Kuliah	

Kolom Verifikasi Soal	
Tanggal dan TandaTangan Dosen	*Tanggal dan Tanda Tangan Kajur*
	

Petunjuk Peserta Ujian

1. *Tuliskan Nama dan NPM dengan lengkap.*
2. *Bacalah dengan seksama, kemudian baru kerjakan, pilih yang mudah.*
3. *Ikuti petunjuk dan tata tertib.*
4. *Selamat mengerjakan*

No	Soal	Bobot	Output / Berkas
1	Studi Kasus & Use Case Diagram: Mengajukan ide aplikasi interaktif (non-CRUD statis) dan merancang Use Case Diagram yang memetakan aktor sistem, fungsionalitas utama, serta relasi use case (*include/extend*) secara tepat.	10	Dokumen Laporan & Diagram (.pdf/.png)
2	Use Case Description (Focus on Dynamic/Multi-Input): Menjabarkan minimal 3 use case utama dalam bentuk tabel terstruktur: aktor, tujuan, *precondition*, *action-reaction flow* (interaksi View respons Controller), skenario alternatif saat validasi input gagal, dan *postcondition*.	10	Dokumen Laporan & Diagram (.pdf/.png)
3	Sequence Diagram (View-Controller Focus): Merancang Sequence Diagram yang secara ketat menggambarkan alur interaksi View (HTML Template) - Controller (Component TS): *event binding*, pemanggilan *handler/method*, mutasi array lokal di memory, kalkulasi *real-time*, dan *re-render* data ke View (tanpa melibatkan model backend/database).	15	Dokumen Laporan & Diagram (.pdf/.png)
4	Component Architecture & Data Mapping: Menyusun tabel pemetaan arsitektur komponen Angular: pohon hierarki komponen (*Component Tree*), pemetaan elemen View-Controller, serta alur data antar-komponen (@Input, @Output / EventEmitter, atau UI State Service).	10	Dokumen Laporan & Diagram (.pdf/.png)
5.1	Implementasi: Dynamic Multi-Input FormArray & Cross-	15	Source code

	Validation: • Mengimplementasikan FormArray dinamis di mana pengguna dapat menambah baris baru (*append*), menghapus baris pada indeks spesifik (*remove at index*), serta memiliki input array bertingkat (*nested items*). • Validasi Lintas-Baris (*Cross-Item Validation*): Controller wajib memvalidasi hubungan antar-baris (misal: larangan duplikasi nilai antar-baris, akumulasi persentase pas 100%, kuota, atau batasan minimal/maksimal baris).		Angular (GitHub / ZIP)
5.2	Implementasi: Direktif *ngFor Tingkat Lanjut (atau @for): • Wajib memanfaatkan Context Variables minimal index (penomoran/id kontrol) dan first/last (penguncian tombol aksi pada baris pertama/terakhir). • Menerapkan fungsi trackBy kustom pada Controller untuk optimasi *rendering* DOM. • Wajib mengimplementasikan perulangan bertingkat (*Nested Loops*) untuk data relasional/sub-item.	15	Source code Angular (GitHub / ZIP)
5.3	Implementasi: Direktif *ngIf & UI State Management (atau @if): • Menerapkan pola *Template Branching* (ng-template #else/#then) untuk memisahkan secara jelas tampilan *Empty State* (saat data kosong) dan *Populated State* (saat data terisi). • Clean Code: Seluruh logika evaluasi boolean yang kompleks wajib diolah sebagai *Getter Method* di Controller, dilarang menuliskan logika panjang di HTML template.	10	Source code Angular (GitHub / ZIP)
6	Presentasi, Live Demo & Technical Code Defense: Demonstrasi aplikasi berjalan saat sesi UTS: pengujian skenario ekstrem/*edge cases* (penambahan banyak baris, hapus acak baris, validasi lintas-baris), serta mempertanggungjawabkan sinkronisasi 100% antara kode Controller dengan Sequence Diagram.	15	Sesi live demo & wawancara kode

PETUNJUK UMUM & KETENTUAN PROYEK

1. Ujian dikerjakan secara individu dalam bentuk perancangan dan implementasi aplikasi web interaktif.
2. Mahasiswa secara bebas mengusulkan (propose) ide studi kasus aplikasi
3. Fokus implementasi berada murni pada lapisan View (Template HTML) dan Controller (Component TypeScript) pada Framework Angular. Lapisan Model (Backend API / Database persistence) belum diwajibkan.
4. Seluruh penamaan class, method handler, dan property pada kode program wajib 100% konsisten dengan rancangan Sequence Diagram.

ASSESSMENT RUBRIC

No	Komponen Penilaian	Bobot	Skor 4 (Sangat Baik)	Skor 3 (Baik)	Skor 2 (Cukup)	Skor 1 (Kurang)
1	Proposal & Use Case Diagram	10%	Ide studi kasus orisinal, interaktif, non-CRUD standar. Diagram mencakup seluruh fungsi utama, aktor, dan relasi (*include/extend*) tepat serta konsisten.	Studi kasus cukup interaktif, diagram use case lengkap dengan sedikit kekurangan minor pada notasi relasi.	Studi kasus mendekati CRUD biasa/statis. Use case belum mencerminkan aksi interaktif atau relasi tidak lengkap.	Studi kasus CRUD dasar/statis biasa, diagram tidak sesuai kaidah UML atau tidak mencerminkan sistem.
2	Deskripsi Use Case (Multi-Input)	10%	Menjabarkan min. 3 use case dinamis secara presisi (aktor, *precondition*, alur aksi-reaksi View-Controller, alternatif validasi baris, dan *postcondition*).	Menjabarkan 3 use case dengan cukup lengkap, alur aksi-reaksi jelas namun penanganan alternatif validasi kurang mendalam.	Hanya sebagian use case dijabarkan atau alur interaksi View-Controller masih ambigu dan bersifat umum.	Deskripsi sangat singkat, format tidak sesuai, atau tidak merefleksikan proses manipulasi multi-input dinamis.
3	Sequence Diagram (View-Controller)	15%	Alur interaksi View (HTML) Controller (TS) runtut & akurat: *event binding*, mutasi *state array in-memory*, evaluasi kondisi, tanpa *layer persistence* (DB).	Alur interaksi View-Controller cukup runtut dan tepat, terdapat minor error pada penamaan pesan atau urutan sinkronisasi.	Alur pesan ambigu/tidak konsisten; masih mencampurkan *layer* Model database atau tidak menggambarkan siklus *event binding*.	Diagram salah konsep UML, tidak mencerminkan interaksi View-Controller, atau tidak dibuat.
4	Component &	10%	Pohon hierarki komponen	Hierarki komponen cukup	Hanya sebagian	Tidak ada pemetaan

	Interaction Mapping		(*Component Tree*) terstruktur; pemetaan View-Controller, *binding* data (@Input, @Output, Service Controller) terdefinisi sangat jelas.	terstruktur; pemetaan *data binding* jelas dengan sedikit ketidakkonsistenan minor.	komponen yang dipetakan; alur komunikasi data antar komponen belum jelas atau terpusat di satu file.	komponen atau arsitektur tidak berbasis modular komponen Angular.
5.1	Implementasi: FormArray & Cross-Validation	15%	Penambahan baris (*append*), penghapusan pada indeks spesifik (*remove at index*), dan validasi lintas-baris (*cross-item*) berfungsi sempurna tanpa *bug*.	Tambah dan hapus baris dinamis berfungsi baik, namun pesan error atau validasi lintas-baris masih memiliki *bug* kecil.	Hanya bisa menambah baris; fungsi hapus indeks tertentu merusak urutan form atau validasi lintas-baris tidak ada.	Form input bersifat statis / *hardcoded*; tidak menerapkan FormArray atau manipulasi array dinamis.
5.2	Implementasi: *ngFor & Optimasi	15%	Menggunakan *context variables* (index, first/last), mengimplementasikan trackBy kustom pada Controller, serta terdapat *nested loop* terstruktur.	Menggunakan index dan fungsi trackBy, namun data array masih satu tingkat (*flat array* / belum *nested*).	Menggunakan *ngFor standar tanpa trackBy dan tanpa pemanfaatan *context variable* (index, first, last).	Sintaks perulangan salah, tidak efisien, atau menyebabkan *rendering error* pada browser DOM.
5.3	Implementasi: UI State & *ngIf	10%	Memisahkan tampilan via ng-template (#else/#then) untuk *Empty State* vs *Populated State*; logika kondisi kompleks rapi di Getter Controller.	Menggunakan *branching else template*, namun logika kondisi boolean masih ditulis panjang langsung di berkas HTML.	Menggunakan *ngIf dasar tanpa penanganan tampilan data kosong (*Empty State*) dan tanpa ng-template.	Logika pengkondisian tidak dinamis, *hardcoded*, atau antarmuka tidak merespons perubahan *state* Controller.

6	Presentasi & Technical Defense	15%	Mampu mendemonstrasikan skenario ekstrem/*edge case* input array dengan lancar, menguasai alur kode, serta sinkron 100% dengan diagram UML.	Presentasi dan demo lancar; mampu menjelaskan kode Controller dengan baik meski ada kendala teknis minor.	Presentasi kurang sistematis; kesulitan menjelaskan korelasi antara diagram UML dengan implementasi kodingan Angular.	Tidak mampu mendemonstrasikan aplikasi, aplikasi gagal berjalan (*error*), atau tidak dapat menjawab pertanyaan teknis.